```
from google.colab import drive
drive.mount('/content/drive')


```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=Tr
```

```
# 2. Add the directory containing 'models.py' to Python's path
import sys
sys.path.append('/content/drive/MyDrive/') # Assuming 'models.py' is directly in MyDrive

# 3. Import the module
import models
from models import *
```

```
import argparse
import os
import time
import shutil

import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import torch.backends.cudnn as cudnn


import torchvision
import torchvision.transforms as transforms

from models import *


global best_prec
use_gpu = torch.cuda.is_available()
print('=> Building model...')



batch_size = 128
model_name = "VGG16_quant_8x8"
model = VGG16_quant_8x8()

print(model)

normalize = transforms.Normalize(mean=[0.491, 0.482, 0.447], std=[0.247, 0.243, 0.262])


train_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=True,
    download=True,
    transform=transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        normalize,
    ]))
trainloader = torch.utils.data.DataLoader(train_dataset, batch_size=batch_size, shuffle=True, num_workers=2)


test_dataset = torchvision.datasets.CIFAR10(
    root='./data',
    train=False,
    download=True,
    transform=transforms.Compose([
        transforms.ToTensor(),
        normalize,
    ]))

testloader = torch.utils.data.DataLoader(test_dataset, batch_size=batch_size, shuffle=False, num_workers=2)


print_freq = 100 # every 100 batches, accuracy printed. Here, each batch includes "batch_size" data points
# CIFAR10 has 50,000 training data, and 10,000 validation data.

def train(trainloader, model, criterion, optimizer, epoch):
    batch_time = AverageMeter()
```

```
    data_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    model.train()

    end = time.time()
    for i, (input, target) in enumerate(trainloader):
        # measure data loading time
        data_time.update(time.time() - end)

        input, target = input.cuda(), target.cuda()

        # compute output
        output = model(input)
        loss = criterion(output, target)

        # measure accuracy and record loss
        prec = accuracy(output, target)[0]
        losses.update(loss.item(), input.size(0))
        top1.update(prec.item(), input.size(0))

        # compute gradient and do SGD step
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # measure elapsed time
        batch_time.update(time.time() - end)
        end = time.time()


        if i % print_freq == 0:
            print('Epoch: [{0}][{1}/{2}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Data {data_time.val:.3f} ({data_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   epoch, i, len(trainloader), batch_time=batch_time,
                   data_time=data_time, loss=losses, top1=top1))


def validate(val_loader, model, criterion ):
    batch_time = AverageMeter()
    losses = AverageMeter()
    top1 = AverageMeter()

    # switch to evaluate mode
    model.eval()

    end = time.time()
    with torch.no_grad():
        for i, (input, target) in enumerate(val_loader):

            input, target = input.cuda(), target.cuda()

            # compute output
            output = model(input)
            loss = criterion(output, target)

            # measure accuracy and record loss
            prec = accuracy(output, target)[0]
            losses.update(loss.item(), input.size(0))
            top1.update(prec.item(), input.size(0))

            # measure elapsed time
            batch_time.update(time.time() - end)
            end = time.time()

            if i % print_freq == 0:  # This line shows how frequently print out the status. e.g., i%5 => every 5 batch
                print('Test: [{0}/{1}]\t'
                  'Time {batch_time.val:.3f} ({batch_time.avg:.3f})\t'
                  'Loss {loss.val:.4f} ({loss.avg:.4f})\t'
                  'Prec {top1.val:.3f}% ({top1.avg:.3f}%)'.format(
                   i, len(val_loader), batch_time=batch_time, loss=losses,
                   top1=top1))

    print(' * Prec {top1.avg:.3f}% '.format(top1=top1))
    return top1.avg


```

```
def accuracy(output, target, topk=(1,)):
    """Computes the precision@k for the specified values of k"""
    maxk = max(topk)
    batch_size = target.size(0)

    _, pred = output.topk(maxk, 1, True, True)
    pred = pred.t()
    correct = pred.eq(target.view(1, -1).expand_as(pred))

    res = []
    for k in topk:
        correct_k = correct[:k].view(-1).float().sum(0)
        res.append(correct_k.mul_(100.0 / batch_size))
    return res


class AverageMeter(object):
    """Computes and stores the average and current value"""
    def __init__(self):
        self.reset()

    def reset(self):
        self.val = 0
        self.avg = 0
        self.sum = 0
        self.count = 0

    def update(self, val, n=1):
        self.val = val
        self.sum += val * n
        self.count += n
        self.avg = self.sum / self.count


def save_checkpoint(state, is_best, fdir):
    filepath = os.path.join(fdir, 'checkpoint.pth')
    torch.save(state, filepath)
    if is_best:
        shutil.copyfile(filepath, os.path.join(fdir, 'model_best.pth.tar'))


def adjust_learning_rate(optimizer, epoch):
    """For resnet, the lr starts from 0.1, and is divided by 10 at 80 and 120 epochs"""
    adjust_list = [70,90]
    if epoch in adjust_list:
        for param_group in optimizer.param_groups:
            param_group['lr'] = param_group['lr'] * 0.1

#model = nn.DataParallel(model).cuda()
#all_params = checkpoint['state_dict']
#model.load_state_dict(all_params, strict=False)
#criterion = nn.CrossEntropyLoss().cuda()
#validate(testloader, model, criterion)
```

```
    )
    (30): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (31): ReLU(inplace=True)
    (32): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (33): QuantConv2d(
      256, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (34): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (35): ReLU(inplace=True)
    (36): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (37): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (38): ReLU(inplace=True)
    (39): QuantConv2d(
      512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
      (weight_quant): weight_quantize_fn()
    )
    (40): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (41): ReLU(inplace=True)
    (42): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (43): AvgPool2d(kernel_size=1, stride=1, padding=0)
  )
  (classifier): Linear(in_features=512, out_features=10, bias=True)
)
```

```
model.features[0] ## Modified layer
```

```
QuantConv2d(
  3, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
)
```

Use the below cells to train a VGGNet16_quant_8x8 model

```
# This cell is from the website

lr = 8e-3


weight_decay = 1e-4
epochs = 100
best_prec = 0

model = model.cuda()
criterion = nn.CrossEntropyLoss().cuda()
optimizer = torch.optim.SGD(model.parameters(), lr=lr, momentum = 0.94, weight_decay=weight_decay) #changed momentum ar
# weight decay: for regularization to prevent overfitting

```

```


if not os.path.exists('result'):
    os.makedirs('result')

fdir = 'result/'+str(model_name)

if not os.path.exists(fdir):
    os.makedirs(fdir)


```

```

for epoch in range(0, epochs):
    adjust_learning_rate(optimizer, epoch)

    train(trainloader, model, criterion, optimizer, epoch)

    # evaluate on test set
    print("Validation starts")
    prec = validate(testloader, model, criterion)

    # remember best precision and save checkpoint
    is_best = prec > best_prec
    best_prec = max(prec,best_prec)
    print('best acc: {:1f}'.format(best_prec))
    if is_best:
      save_checkpoint({
        'epoch': epoch + 1,
```

```
        'state_dict': model.state_dict(),
        'best_prec': best_prec,
        'optimizer': optimizer.state_dict(),
    }, is_best, fdir)

    if prec > 90: break

```

```
 * Prec 87.480%
best acc: 89.100000
Epoch: [64][0/391]      Time 0.136 (0.136)      Data 0.106 (0.106)      Loss 0.1448 (0.1448)    Prec 93.750% (93.750%)
Epoch: [64][100/391]    Time 0.029 (0.026)      Data 0.008 (0.005)      Loss 0.0324 (0.0892)    Prec 99.219% (97.014%)
Epoch: [64][200/391]    Time 0.028 (0.026)      Data 0.008 (0.005)      Loss 0.1469 (0.0897)    Prec 94.531% (96.984%)
Epoch: [64][300/391]    Time 0.028 (0.025)      Data 0.007 (0.004)      Loss 0.1034 (0.0944)    Prec 95.312% (96.750%)
Validation starts
Test: [0/79]    Time 0.094 (0.094)      Loss 0.2602 (0.2602)    Prec 93.750% (93.750%)
 * Prec 89.150%
best acc: 89.150000
Epoch: [65][0/391]      Time 0.136 (0.136)      Data 0.107 (0.107)      Loss 0.0673 (0.0673)    Prec 98.438% (98.438%)
Epoch: [65][100/391]    Time 0.029 (0.025)      Data 0.008 (0.005)      Loss 0.0452 (0.0910)    Prec 98.438% (96.720%)
Epoch: [65][200/391]    Time 0.027 (0.025)      Data 0.007 (0.004)      Loss 0.0986 (0.0935)    Prec 96.094% (96.720%)
Epoch: [65][300/391]    Time 0.029 (0.025)      Data 0.008 (0.004)      Loss 0.0660 (0.0971)    Prec 96.875% (96.566%)
Validation starts
Test: [0/79]    Time 0.094 (0.094)      Loss 0.1738 (0.1738)    Prec 96.094% (96.094%)
 * Prec 88.900%
best acc: 89.150000
Epoch: [66][0/391]      Time 0.137 (0.137)      Data 0.107 (0.107)      Loss 0.0656 (0.0656)    Prec 96.875% (96.875%)
Epoch: [66][100/391]    Time 0.028 (0.026)      Data 0.007 (0.005)      Loss 0.1140 (0.0937)    Prec 96.875% (96.682%)
Epoch: [66][200/391]    Time 0.026 (0.025)      Data 0.005 (0.005)      Loss 0.1197 (0.1011)    Prec 96.094% (96.459%)
Epoch: [66][300/391]    Time 0.020 (0.025)      Data 0.001 (0.004)      Loss 0.1615 (0.1012)    Prec 96.094% (96.444%)
Validation starts
Test: [0/79]    Time 0.096 (0.096)      Loss 0.2893 (0.2893)    Prec 90.625% (90.625%)
 * Prec 88.510%
best acc: 89.150000
Epoch: [67][0/391]      Time 0.135 (0.135)      Data 0.105 (0.105)      Loss 0.0553 (0.0553)    Prec 97.656% (97.656%)
Epoch: [67][100/391]    Time 0.027 (0.025)      Data 0.006 (0.005)      Loss 0.1017 (0.0938)    Prec 95.312% (96.604%)
Epoch: [67][200/391]    Time 0.029 (0.025)      Data 0.008 (0.004)      Loss 0.0919 (0.0898)    Prec 95.312% (96.852%)
Epoch: [67][300/391]    Time 0.021 (0.025)      Data 0.000 (0.004)      Loss 0.0860 (0.0950)    Prec 97.656% (96.649%)
Validation starts
Test: [0/79]    Time 0.094 (0.094)      Loss 0.4153 (0.4153)    Prec 92.188% (92.188%)
 * Prec 88.800%
best acc: 89.150000
Epoch: [68][0/391]      Time 0.145 (0.145)      Data 0.108 (0.108)      Loss 0.0323 (0.0323)    Prec 99.219% (99.219%)
Epoch: [68][100/391]    Time 0.029 (0.026)      Data 0.008 (0.004)      Loss 0.0652 (0.0888)    Prec 98.438% (97.022%)
Epoch: [68][200/391]    Time 0.027 (0.025)      Data 0.007 (0.004)      Loss 0.0402 (0.0880)    Prec 99.219% (96.949%)
Epoch: [68][300/391]    Time 0.028 (0.025)      Data 0.008 (0.004)      Loss 0.0531 (0.0912)    Prec 98.438% (96.846%)
Validation starts
Test: [0/79]    Time 0.092 (0.092)      Loss 0.3118 (0.3118)    Prec 89.844% (89.844%)
 * Prec 88.710%
best acc: 89.150000
Epoch: [69][0/391]      Time 0.137 (0.137)      Data 0.107 (0.107)      Loss 0.0679 (0.0679)    Prec 97.656% (97.656%)
Epoch: [69][100/391]    Time 0.026 (0.026)      Data 0.006 (0.005)      Loss 0.0509 (0.0816)    Prec 98.438% (97.184%)
Epoch: [69][200/391]    Time 0.026 (0.025)      Data 0.005 (0.004)      Loss 0.0950 (0.0863)    Prec 98.438% (96.964%)
Epoch: [69][300/391]    Time 0.031 (0.025)      Data 0.010 (0.004)      Loss 0.1033 (0.0885)    Prec 95.312% (96.883%)
Validation starts
Test: [0/79]    Time 0.095 (0.095)      Loss 0.3253 (0.3253)    Prec 93.750% (93.750%)
 * Prec 89.450%
best acc: 89.450000
Epoch: [70][0/391]      Time 0.139 (0.139)      Data 0.109 (0.109)      Loss 0.0552 (0.0552)    Prec 96.875% (96.875%)
Epoch: [70][100/391]    Time 0.032 (0.027)      Data 0.011 (0.005)      Loss 0.0836 (0.0666)    Prec 96.094% (97.633%)
Epoch: [70][200/391]    Time 0.029 (0.026)      Data 0.009 (0.005)      Loss 0.1219 (0.0608)    Prec 95.312% (97.897%)
Epoch: [70][300/391]    Time 0.030 (0.026)      Data 0.009 (0.005)      Loss 0.0989 (0.0569)    Prec 96.875% (98.004%)
Validation starts
Test: [0/79]    Time 0.094 (0.094)      Loss 0.2839 (0.2839)    Prec 91.406% (91.406%)
 * Prec 90.510%
best acc: 90.510000
```

```
cp -r  /content/result/VGG16_quant_8x8 /content/drive/MyDrive/result

```

```
PATH = '/content/drive/MyDrive/result/VGG16_quant_8x8/model_best.pth.tar'
checkpoint = torch.load(PATH)
model.load_state_dict(checkpoint['state_dict'])
device = torch.device("cuda")

model.cuda()
model.eval()

test_loss = 0
correct = 0

with torch.no_grad():
    for data, target in testloader:
        data, target = data.to(device), target.to(device) # loading to GPU
        output = model(data)
        pred = output.argmax(dim=1, keepdim=True)
```

```
        correct += pred.eq(target.view_as(pred)).sum().item()

test_loss /= len(testloader.dataset)

print('\nTest set: Accuracy: {}/{} ({:.0f}%)\n'.format(
        correct, len(testloader.dataset),
        100. * correct / len(testloader.dataset)))
```

```
Test set: Accuracy: 9051/10000 (91%)
```

```
class SaveOutput:
    def __init__(self):
        self.outputs = []
    def __call__(self, module, module_in):
        self.outputs.append(module_in)
    def clear(self):
        self.outputs = []

######### Save inputs from selected layer ##########
save_output = SaveOutput()
i = 0
count = 0
for layer in model.modules():
    i = i+1
    if isinstance(layer, QuantConv2d):
        count += 1
        print(i,"-th layer prehooked \n", layer, count) #'count' here is just for me to check the index of save_output.
        layer.register_forward_pre_hook(save_output)
####################################################

dataiter = iter(testloader)
images, labels = next(dataiter)
images = images.to(device)
images_abs = torch.abs(images) # taking absolute value of image input since our feature map activations are assumed to
out = model(images_abs)
```

```
 64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 2
12 -th layer prehooked
QuantConv2d(
 64, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 3
16 -th layer prehooked
QuantConv2d(
 128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 4
21 -th layer prehooked
QuantConv2d(
 128, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 5
25 -th layer prehooked
QuantConv2d(
 256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 6
29 -th layer prehooked
QuantConv2d(
 256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 7
34 -th layer prehooked
QuantConv2d(
 256, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 8
38 -th layer prehooked
QuantConv2d(
 8, 8, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
 (weight_quant): weight_quantize_fn()
) 9
41 -th layer prehooked
QuantConv2d(
 8, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
```

```
50 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 12
54 -th layer prehooked
 QuantConv2d(
  512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False
  (weight_quant): weight_quantize_fn()
) 13
```

```
weight_q = model.features[0].weight_q
w_alpha = model.features[0].weight_quant.wgt_alpha
w_bit = 4

weight_int = weight_q / (w_alpha / (2**(w_bit-1)-1))
#print(weight_int)
```

```
act = images_abs
act_alpha  = model.features[0].act_alpha
act_bit = 4
act_quant_fn = act_quantization(act_bit)

act_q = act_quant_fn(act, act_alpha)

act_int = act_q / (act_alpha / (2**act_bit-1))
print(torch.all(act_int >= 0)) ## all are positive
```

```
tensor(True, device='cuda:0')
```

```
conv_int = torch.nn.Conv2d(in_channels = 3, out_channels=64, kernel_size = 3, padding=1)
conv_int.weight = torch.nn.parameter.Parameter(weight_int)
relu = torch.nn.ReLU()
conv_int.bias = model.features[0].bias
output_int = conv_int(act_int) # The most important thing
output_recovered = output_int * (act_alpha / (2**act_bit-1)) * (w_alpha / (2**(w_bit-1)-1))
output_recovered_relu = relu(output_recovered)
#print(output_recovered)
```

```
print(save_output.outputs[1][0].shape)
difference = abs(save_output.outputs[1][0] - output_recovered_relu )
print(difference.mean())  ## It should be small, e.g.,2.3 in my trainned model
```

```
torch.Size([128, 64, 32, 32])
tensor(0.9344, device='cuda:0', grad_fn=<MeanBackward0>)
```

```
import torch
import torch.nn.functional as F

# ------------------------------------------------
# 1. Conv hyperparameters – EDIT THESE TO MATCH YOUR LAYER
# ------------------------------------------------
stride = 1        # or (1, 1)
padding = 1       # use 0 if you had no padding
dilation = 1
groups = 1

use_bias = False  # set True and define bias_int if your conv has bias
bias_int = None   # e.g. torch.randint(..., size=(64,), dtype=torch.int16)

# If you do integer scaling (e.g., >> shift_bits after accumulation)
apply_shift = False
shift_bits = 3    # example: right shift by 3

# ------------------------------------------------
# 2. Basic sanity prints
# ------------------------------------------------
print("act_int.shape    :", act_int.shape)     # [128, 3, 32, 32]
print("weight_int.shape :", weight_int.shape)  # [64, 3, 3, 3]
print("output_int.shape :", output_int.shape)  # [128, 64, 32, 32]

batch = 5
out_c = 1
h = 0
w = 0

# ------------------------------------------------
# 3. Float conv2d reference using act_int & weight_int
# ------------------------------------------------
act_f = act_int.to(torch.float32)
weight_f = weight_int.to(torch.float32)
```

```
bias_f = None
if use_bias and bias_int is not None:
    bias_f = bias_int.to(torch.float32)

out_f = F.conv2d(
    act_f, weight_f,
    bias=bias_f,
    stride=stride,
    padding=padding,
    dilation=dilation,
    groups=groups,
)

print("F.conv2d reference (float):", out_f[batch, out_c, h, w].item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())

# ----------------------------------------------------
# 4. Manual integer convolution using act_int & weight_int
# ----------------------------------------------------
# We need to mimic the same padding as conv2d.
# F.pad pads in the order (left, right, top, bottom).
if isinstance(padding, int):
    pad_left = pad_right = pad_top = pad_bottom = padding
else:
    # if you used tuple padding, adjust this section
    pad_left = pad_right = padding[1] if isinstance(padding, tuple) else padding
    pad_top  = pad_bottom = padding[0] if isinstance(padding, tuple) else padding

act_padded = F.pad(act_int, (pad_left, pad_right, pad_top, pad_bottom))

# Kernel size
_, in_channels, kH, kW = weight_int.shape

# For stride=1, dilation=1, the input top-left index for output (h, w) is:
h_in = h * stride
w_in = w * stride

# Extract 3x3 (or kH x kW) patch from padded input
patch_int = act_padded[batch, :, h_in:h_in + kH, w_in:w_in + kW]   # [Cin, kH, kW]
kernel_int = weight_int[out_c]                                     # [Cin, kH, kW]

# Integer accumulation in int32 to avoid overflow
acc_int32 = (patch_int.to(torch.int32) * kernel_int.to(torch.int32)).sum()

if use_bias and bias_int is not None:
    acc_int32 = acc_int32 + int(bias_int[out_c].item())

print("Manual integer accumulator :", acc_int32.item())

# ----------------------------------------------------
# 5. Optional: apply integer scaling / shifting and clamp to int16
# ----------------------------------------------------
if apply_shift:
    # arithmetic right shift for signed values
    acc_shifted = acc_int32 >> shift_bits
else:
    acc_shifted = acc_int32

# Clamp to int16 range
acc_int16 = torch.clamp(acc_shifted, -32768, 32767).to(torch.int16)

print("Manual quantized int16     :", acc_int16.item())
print("Stored output_int          :", output_int[batch, out_c, h, w].item())
```

```
act_int.shape    : torch.Size([128, 3, 32, 32])
weight_int.shape : torch.Size([64, 3, 3, 3])
output_int.shape : torch.Size([128, 64, 32, 32])
F.conv2d reference (float): 11.0
Stored output_int          : 11.0
Manual integer accumulator : 11
Manual quantized int16     : 11
Stored output_int          : 11.0
```

```
batch = 5
X = act_padded[batch,:,:,:]  # pick only one input C_in, ni, nj
w_int = torch.reshape(weight_int, (weight_int.size(0), weight_int.size(1), -1))  # out_ch = 64, in_ch = 3, kij = 9
W = w_int
out_int = output_int[batch]
out_int = torch.reshape(out_int, (out_int.size(0), -1))

```

```
print(X.shape)
print(W.shape)
```

```
torch.Size([3, 34, 34])
torch.Size([64, 3, 9])
```

How are the activation and weight txt files created?

- Each row in the mac represents a feature in the output feature mape
- Each column is the output channel of that feature in the output feature map
- The time for loading the weights and activations is equal to 3, because we have a set of 3 input channels to sum over to produce the output feature map
- Each row in the Mac has to be the window of convolution over the activatino map which maps to the respective output feature in the output feature map, so row 1 is o_nij = 0 and row 8 is o_nij = 7. The time for each PE represents the index of the activation that is multiplied with the respective kij for that convolution operation.
- The weight file contains rows that represent each kij from kij = 0 to kij = 8 for each of the output channels, 0 to 8.

The below cell implements the details discussed above so that the correct operations are performed in each of the PE's of the output stationary design

```
batch = 5
in_channels = 3
bit_precision = 4
h = 3
k_size = 3
array_size = 8
kij = 9

for i in range(in_channels):
  fname = f"activation_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for j in range(h):
    for k in range(h):
      for l in range(array_size):
        #print(round(X[i,j,7-l+k].item()))
        X_bin = '{0:04b}'.format(round(X[i,j,7-l+k].item()))
        for m in range(bit_precision):
              file.write(X_bin[m])
      file.write('\n')

  file.close()

w_check = []

for i in range(in_channels):
  fname = f"weight_in_ch{i}.txt"
  file = open(fname, 'w') #write to file
  for ker in range(kij):
    for j in range(array_size):
      if (7-j == 2):
        w_check.append(W[7-j,i,ker].item())
        #print(f"o_channel: {7-j}, in_channel {i}, kernel {ker}, value {w_check[-1]}")
      if (round(W[7-j,i,k].item()) >= 0):
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item()))
      else:
          W_bin = '{0:04b}'.format(round(W[7-j,i,ker].item())+16)
      for k in range(bit_precision):
          file.write(W_bin[k])
    file.write('\n')
  file.close()

```

```
out_int.shape #out_ch, o_nij
```

```
torch.Size([64, 1024])
```

How is the output txt file generated:

- Each row is a different O_nij, we note that in our testbench we test the correctness of each output feature from the 7th output feature down to the 0th output feature map, so the first row in the output txt file is output feature 7 going down ot output feature 0. We only tested the correctness of the first 8 output features for the first 8 output channels out of the 64 output channels.

```
o_nij = 8
o_chan = 8
```

```
bit_precision = 16

fname = "out.txt"
file = open(fname, 'w') #write to file
out_int = relu(out_int) #relu(out_int)
for i in range(o_nij):
  for o in range(o_chan):
    #out_int[7-o,7-i]
    if (7-o == 1 and 7-i == 0): print(out_int[7-o,7-i].item())
    if (out_int[7-o,7-i].item() >= 0):
        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item()))
    else:
        Out_bin = '{0:016b}'.format(round(out_int[7-o,7-i].item())+65536)
    for k in range(bit_precision):
        file.write(Out_bin[k])
  file.write('\n')
file.close()
```

```
11.0
```

```
# print(out_int[1,1])
# print(out_int.shape)
# print(X.shape)
# print(W.shape)
```

```
# (X[:,0:3,1:4]*W[1,:,:].reshape(3,3,3)).sum() #equivalent  to out_ch = 1 and o_ij = 0
```

```
# torch.set_printoptions(profile="full")
# print(w_check[18:])
# print(W[2,2,:].reshape(3,3)) #o_ch = 2, in_ch = 2, 0:9
# weight_int[2,2,:,:]
```